Dr. Dobb's Journal November 2007

Apply Critical Sections Consistently

Guaranteeing mutual exclusion on shared variables

By Herb Sutter

Herb is a software architect at Microsoft and chair of the ISO C++ Standards committee. He can be contacted at www.gotw.ca.

The critical section is our One True Tool for guaranteeing mutual exclusion on mutable shared variables. Table 1 summarizes several common ways to express exclusive critical sections (see last month's column [1] for details). A useful way to think about the synchronization in your program is as a daisy-chain of these "release-acquire" handoffs that stitch all the threads' critical sections together into some linear order, so that each one "acquires" the cumulative work done and is "released" by all the others that preceded it.

Like most tools, these must be applied consistently, and with the intended meanings. A program must ensure that every use of a mutable shared object is properly protected using exactly one of these mechanisms at any given point in its lifetime. Chaos can erupt if code tries to avoid or invert these meanings (e.g., trying to abuse taking a lock or reading an atomic variable as a "critical section exit" operation; see Example 3), or tries to use inconsistent synchronization techniques at the same time.

Let's consider some examples to illustrate the proper and improper uses of critical sections, to get us used to looking for where the critical sections' synchronization points should appear in the code.

Synchronization Type	To Enter a Critical Section	To Exit a Critical Section	Notes
Locks	Acquire lock	Release lock	Preferred, despite their drawbacks [2]
Condition variables	Wait for cv	Notify cv	Useful with locks to express an efficient wait point in the middle of a locked section
Semaphores	Acquire semaphore	Release semaphore	Use judiciously; significant care required
Ordered atomics (e.g., Java/.NET volatile, C++0x atomic)	Read from variable *	Write to variable **	Use judiciously, significant care required
Unordered atomics and explicit fences (e.g., aligned integers and Linux mb or Win32 MemoryBarrier)	Read from variable followed by fence***	Fence followed by write to variable***	Avoid, difficult to use correctly and usually nonportable

* *or equivalent, such as calling a compare-and-swap function having at least acquire semantics*

** *or equivalent, such as calling a compare-and-swap function having at least release semantics*

*** *when in doubt, use a full fence*

Table 1: Common ways to express critical sections.

Example 1: One Producer, Many Consumers, Using Locks

Imagine we have a single Producer thread that produces a number of tasks for Consumer threads to perform. The Producer pushes the tasks into a sha *queue*, and finally pushes a special *done* sentinel to mark the end of the work. The *queue* object is protected at all times using mutex *mut*, which the

Producer holds only as long as necessary for a single *push*, so that Consumers can start executing their tasks concurrently with the Producer. For further efficiency, to avoid making the Consumers busy-wait whenever the queue is initially or temporarily empty, the Producer will also signal condition variable *cv* every time something has been added to the queue:

```
// One Producer thread
//
while( ThereAreMoreTasks() ) {
  task = AllocateAndBuildNewTask();
  mut.lock();           // enter critical section
  queue.push( task );   // add new task
  mut.unlock(); // exit critical section
  cv.notify();
}
mut.lock();             // enter critical section
queue.push( done );     // add sentinel value; that's all folks
mut.unlock();   // exit critical section
cv.notify();
```

On the consumption side, the Consumer threads each pull individual tasks off the completed queue. Here, *myTask* is an ordinary variable local to each Consumer:

```
// K Consumer threads
//
myTask = null;
while( myTask != done ) {
  mut.lock();   // enter critical section
  while( queue.empty() )        // if nothing's there, to avoid busy-w
    cv.wait( mut );     // release and reenter critical section
  myTask = queue.first();       // take a copy of the task
  if( myTask != done )  // remove it from queue unless it was the sent
    queue.pop();        // which we want to leave there for others to
  mut.unlock(); // exit critical section
  if( myTask != done )
    DoWork( myTask );
}
```

All of these critical sections are easy to see. This code simply protects the data structure using the same mutex for its entire shared lifetime, and it's easy to check that you did it right—just look to make sure the code only refers to *queue* when inside some critical section that holds a lock on *mut*.

The primary expression of critical sections is the explicit locking done on the mutex. The condition variable is just icing on the cake—an optimization that lets us express an efficient wait point in the middle of a locked section so that the Consumer race cars don't have to expend needless energy spinning their tires (in this case, their *lock/unlock* loop) while waiting for the Producer to give them the green light.

Example 2: One Producer, Many Consumers, Using Lock-Free Mailboxes

Next, let's consider a similar example, but instead of locks ,we'll use ordered atomic variables to synchronize the Producer-to-Consumers handoff...and synchronization at all for interConsumers contention.

The idea in this second approach is to use a set of ordered atomic variables (e.g., Java/.NET *volatile*, proposed C++0x *atomic*), one for each Consumer, that will serve as "mailbox" slots for the Consumers individually. Each mail slot can hold exactly one item at a time. As the Producer produces each new task, he puts it in the next mailbox that doesn't already contain an item using an atomic write; this lets the Consumers run concurrently with the Producer, as they check their slots and perform work while the Producer is still manufacturing and assigning later tasks. For further efficiency, to avoid making the Consumer busy-wait whenever its slot is initially or temporarily empty, we'll also give each Consumer a semaphore *sem* that the Producer will signal every time it puts a task into the Consumer's mail slot.

Finally, once all the tasks have been assigned, the Producer starts to put "done" dummy tasks into mailboxes that don't already contain an item until every mailbox has received one. Here's the code:

```
// One Producer thread: Changes any box from null to nonnull
//
curr = 0;  // keep a finger on the current mailbox

// Phase 1: Build and distribute tasks (use next available box)
while( ThereAreMoreTasks() ) {
  task = AllocateAndBuildNewTask();
  while( slot[curr] != null )   // acquire null: look for next
        curr = (curr+1)%K;              // empty slot
  slot[curr] = task;            // release nonnull: "You have mail!"
  sem[curr].signal();
}

// Phase 2: Stuff the mailboxes with"done" signals
numNotified = 0;
while( numNotified < K ) {
  while( slot[curr] == done )   // acquire: look for next not-yet-
       curr = (curr+1)%K;               // notified slot
  if( slot[curr] == null ) {    // acquire null: check that the
                                        // mailbox is empty
    slot[curr] = done;          // release done: write sentinel
    sem[curr].signal();
    ++numNotified;
  }
}
```

On the other side of the mailbox wall, each Consumer checks only its own mailbox. If its slot holds a new task, the Consumer takes the task, resets its mail slot to "empty," and then processes the task. If the mailbox holds a "done" dummy task, the Consumer knows it can stop.

```
// K Consumer threads (mySlot = 0..K-1):
// Each changes its own box from non-null to null
//
myTask = null;
while( myTask != done ) {
  while( (myTask = slot[mySlot]) == null )      // acquire nonnull,
    sem[mySlot].wait(); // without busy-waiting
  if( myTask != done ) {
    slot[mySlot] = null;        // release null: tell
                                // him we took it
```

```
    DoWork( myTask );
  }
}
```

A few notes: First, you can easily generalize this to replace each slot with a bounded queue; the previous example just expresses a bounded queue with a maximum size of 1. Second, this Producer code can needlessly busy-wait when it's trying to write a new task (or the remaining *done* sentinels) and there are no free slots. This can be addressed by adding another semaphore, which

the Producer waits for when it has no free slots and that is signaled by each Consumer when it resets its slot to null. I've left that logic out for now to keep the example clearer.

In Example 2, it's not quite as easy to check that we did it right as it was when we used locks back in Example 1, but we can reason about the code to convince ourselves that it is correct. First, note how each Consumer's read of its slot "acquires" whatever value the Producer last wrote there, and the Producer's read of each slot "acquires" whatever value the corresponding Consumer last wrote there. We avoid conflicts because while the slot is null, it's owned by the Producer (only the Producer can change a slot from null to non-null); and while it's nonnull, it's owned by its corresponding Consumer (only the Consumer can change its slot from nonnull to null).

Example 3: Don't Try to Turn Critical Sections Inside Out

Although every entry/exit of a critical section implies a use of locks or atomics, not every use of locks or atomics correctly expresses a critical section. Consider this evil, smelly code of questionable descent, where *x* is initially zero (Note: this example is drawn from [3]):

```
// Thread 1
x = 1;  // a
mut.lock();     // b
... etc. ...
mut.unlock();

// Thread 2: wait for Thread 1 to take the lock, then use x
while( mut.trylock() )  // try to take the lock...
  mut.unlock();         // ... if we succeeded, unlock and loop
r2 = x;         // not guaranteed to see the value 1!
```

This programmer is certainly nobody we would know or associate with. But what is he doing? He is trying to abuse the fact that locking is a global operation, and is visible in principle to all threads.

Specifically, a thread can use a *trylock*-like facility to find out whether some other thread currently holds the lock, by trying to acquire the lock and seeing if that attempt succeeded or failed. Pretty much every threading library has a way to determine if someone else has entered a locked critical section: In Java, you can use *Lock.tryLock*; on Windows and .NET, there's *Monitor.TryEnter* or *WaitOne* with a timeout; in proposed C++0x, *try_lock* or *timed_lock*; and in pthreads, *pthread_mutex_trylock* and *pthread_mutex_timedlock*.

Thread 2, which we might now refer to as Machiavelli, doesn't really want the lock. Machiavelli only wants to try to eavesdrop on Thread 1 in the next room, listening with a *trylock* glass against the wall until he hears the telltale sound that means Thread 1 got to line *b*, and therefore presumably has already set *x*.

The most obvious red flag in this code is that the read and write of *x* are outside critical sections; that is, while we don't hold a lock on *mut*. That technique only works when there's enough synchronization to hand off an object from one thread to another (it belongs to one thread before the synchronization, and another thread after the synchronization), and there isn't enough synchronization here to do that. Let's see why.

The problem is that this anti-idiom is trying to abusively invert the usual meanings of a locked section. Thread 2 is abusively trying to use Thread 1's entering a critical section as a release event, which it isn't. Remember, entering a critical section is an acquire event, and leaving a critical section is a release event. On this, the whole world depends, as we saw last month [1]. In particular, compiler and processor optimizations are guaranteed to respect normal critical section boundaries, and therefore not change the semantics of correctly synchronized code; specifically, they respect the rule that code can be reordered to move into, but not out of, a critical section. In the context of Thread 1, that means an optimizer or processor is free to actually execute line *a* after line *b*... and therefore, Thread 2 could well see a value of 0 for *x,* despite its attempts to abuse the global visibility of locking.

In reality, the code may happen to work all the time on a given system that doesn't happen to reorder lines *a* and *b*. The original programmer might be long gone by the time some other poor sod tries to port it to a new compiler or platform that does manifest the race, and gets to experience the joy of debugging the intermittent problem over a holiday weekend. (Note that, besides the issues we've discussed, this code has other potential problems; for example, Thread 2 can wait forever if Thread 1's lock is taken too early.)

Treat a critical section as you would treat another person: Turning either inside out is at least cruel, and usually illegal. For all the sensational, lurid, and unsavory details, see [3].

Summary

Apply critical sections consistently to protect shared objects: Enter a critical section by taking a lock or reading from an ordered *atomic* variable; and exit a critical section by releasing a lock or writing to an ordered *atomic* variable. Never pervert these meanings; in particular, don't abuse *trylock* to (try to) make a lock acquire in another thread act like the end of a critical section.

Notes

[1] H. Sutter. "Use Critical Sections (Preferably Locks) to Eliminate Races," *Dr. Dobb's Journal*, October 2007.

[2] H. Sutter. "The Trouble With Locks," *C/C++ Users Journal,* March 2005. Available at http://gotw.ca/publications/mill36.htm.

[3] H. Boehm. "Reordering Constraints for Pthread-Style Locks," *Proceedings of the 12th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'07)*, March 2007.